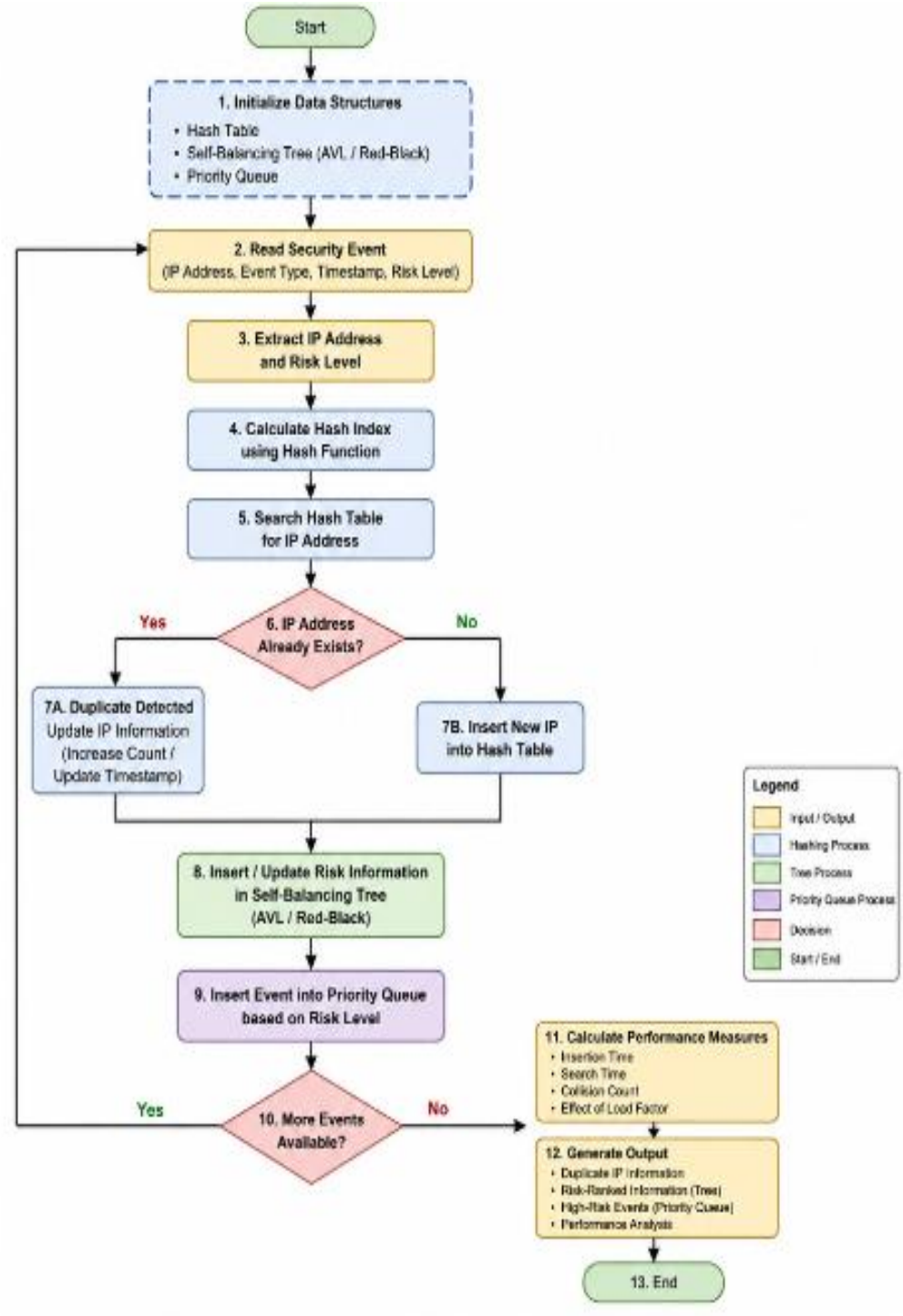
# ASSIGNMENT

## Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

**FLOW CHART:**



**PSEUDO CODE :**

**1. Data Structures Overview :**

```
struct Event:
   ip_address: string
   timestamp: long
   event_type: enum {LOGIN_FAIL, LOGIN_SUCCESS, PORT_SCAN,
ACCESS_REQUEST, ...}
   risk_score: int          // computed from indicators
   indicators: list<string>

struct IPRecord:
   ip_address: string
   frequency_count: int
   last_seen: long
   total_risk_score: int
   event_history: list<Event>
```

**2. Hash Table — Duplicate/Frequency Detection (O(1) avg) :**

```
CLASS HashTable_Chaining:
   buckets: array of LinkedList<IPRecord>
   size: int
   capacity: int
   load_factor_threshold = 0.75

   FUNCTION hash(ip_address):
      RETURN polynomial_hash(ip_address) MOD capacity

   FUNCTION insertOrUpdate(ip_address, event):
      idx = hash(ip_address)
      IF load_factor() > load_factor_threshold:
         resize_and_rehash()
         idx = hash(ip_address)          // recompute after resize

      node = buckets[idx].find(ip_address)
      IF node EXISTS:
         node.frequency_count += 1
         node.last_seen = event.timestamp
         node.total_risk_score += event.risk_score
         node.event_history.append(event)
         RETURN node.frequency_count      // caller checks brute-force threshold
      ELSE:
         newRecord = IPRecord(ip_address, 1, event.timestamp, event.risk_score, [event])
         buckets[idx].append(newRecord)
         size += 1
         RETURN 1
```

```
    FUNCTION lookup(ip_address):              // O(1) average
        idx = hash(ip_address)
        RETURN buckets[idx].find(ip_address)  // NULL or IPRecord

    FUNCTION load_factor():
        RETURN size / capacity

    FUNCTION resize_and_rehash():
        old_buckets = buckets
        capacity = capacity * 2
        buckets = new array[capacity] of empty LinkedList
        size = 0
        FOR each list IN old_buckets:
            FOR each record IN list:
                idx = hash(record.ip_address)
                buckets[idx].append(record)
                size += 1
```

**Alternative: Open Addressing (for comparison benchmark):**

```
CLASS HashTable_OpenAddressing:
    table: array of IPRecord (or TOMBSTONE / EMPTY)
    capacity, size

    FUNCTION probe(ip_address, i):
        // Double hashing to reduce clustering under skew
        RETURN (hash1(ip_address) + i * hash2(ip_address)) MOD capacity

    FUNCTION insertOrUpdate(ip_address, event):
        IF load_factor() > 0.7: resize_and_rehash()
        i = 0
        LOOP:
            idx = probe(ip_address, i)
            IF table[idx] == EMPTY OR table[idx] == TOMBSTONE:
                table[idx] = new IPRecord(...)
                size += 1; RETURN
            IF table[idx].ip_address == ip_address:
                update table[idx]; RETURN
            i += 1

    FUNCTION lookup(ip_address):
        i = 0
        LOOP:
            idx = probe(ip_address, i)
            IF table[idx] == EMPTY: RETURN NULL
            IF table[idx] != TOMBSTONE AND table[idx].ip_address == ip_address:
                RETURN table[idx]
            i += 1
            IF i > capacity: RETURN NULL
```

**3. Self-Balancing BST (Red-Black Tree) — Ordered Risk Index:**

```
ENUM Color { RED, BLACK }

struct RBNode:
    key: (risk_score, ip_address)
    ip_record: pointer to IPRecord
    color: Color
    left, right, parent: RBNode

CLASS RedBlackTree:
    root: RBNode

    FUNCTION insert(key, ip_record):
        node = new RBNode(key, ip_record, RED)
        bst_insert(node)            // standard BST insert by key
        fix_insert(node)            // rebalance + recolor

    FUNCTION fix_insert(node):
        WHILE node.parent.color == RED:
            uncle = get_uncle(node)
            IF uncle.color == RED:
                node.parent.color = BLACK
                uncle.color = BLACK
                node.parent.parent.color = RED
                node = node.parent.parent
            ELSE:
                IF node is "triangle" configuration:
                    rotate toward node to make it a "line"
                node.parent.color = BLACK
                node.parent.parent.color = RED
                rotate at grandparent (left or right)
        root.color = BLACK

    FUNCTION delete(key):
        // standard RB-delete with double-black fixup
        locate node; splice out; fix_delete(replacement)

    FUNCTION rangeQuery(low_score, high_score):
        result = []
        in_order_traverse_with_pruning(root, low_score, high_score, result)
        RETURN result   // O(log n + k), k = results in range

    FUNCTION update_risk(ip_address, old_score, new_score):
        delete(old_score, ip_address)
        insert(new_score, ip_address)   // O(log n)
```

**4. Priority Queue — Real-Time Risk Surfacing (Max-Heap):**

```
CLASS MaxHeap_PriorityQueue:
    heap: array of Event
    size: int

    FUNCTION insert(event):
        heap[size] = event
        i = size
        size += 1
        WHILE i > 0 AND heap[parent(i)].risk_score < heap[i].risk_score:
            swap(heap[i], heap[parent(i)])
            i = parent(i)

    FUNCTION extractMax():              // highest-risk event, O(log n)
        IF size == 0: RETURN NULL
        top = heap[0]
        heap[0] = heap[size - 1]
        size -= 1
        sift_down(0)
        RETURN top

    FUNCTION sift_down(i):
        LOOP:
            l = left(i); r = right(i); largest = i
            IF l < size AND heap[l].risk_score > heap[largest].risk_score: largest = l
            IF r < size AND heap[r].risk_score > heap[largest].risk_score: largest = r
            IF largest == i: BREAK
            swap(heap[i], heap[largest])
            i = largest

    FUNCTION peek():
        RETURN heap[0]      // O(1) — always the current top threat
```

**5. Main Event Processing Pipeline:**

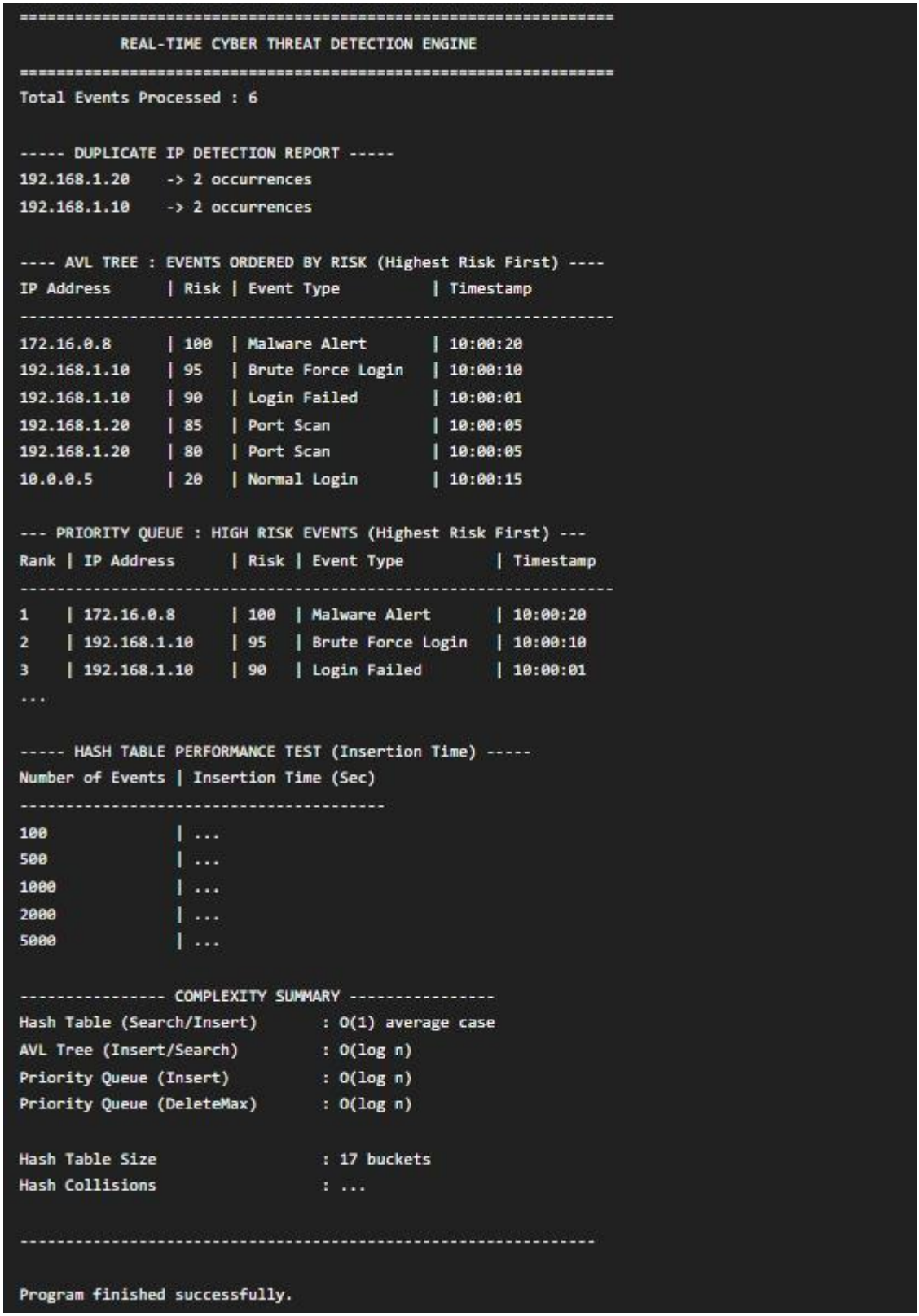
```
FUNCTION processEvent(event):
    // Step 1: risk scoring
    event.risk_score = computeRiskScore(event.indicators, event.event_type)

    // Step 2: duplicate / frequency detection — O(1) average
    freq = hashTable.insertOrUpdate(event.ip_address, event)
    IF freq >= BRUTE_FORCE_THRESHOLD:
        flag_as_brute_force(event.ip_address)
        event.risk_score += ESCALATION_BONUS

    // Step 3: maintain ordered risk index — O(log n)
    record = hashTable.lookup(event.ip_address)
    IF record.previous_score EXISTS:
```

```
        rbTree.update_risk(event.ip_address, record.previous_score, record.total_risk_score)
    ELSE:
        rbTree.insert((record.total_risk_score, event.ip_address), record)

    // Step 4: surface high-risk events — O(log n)
    IF event.risk_score >= ALERT_THRESHOLD:
        priorityQueue.insert(event)

FUNCTION mainLoop(event_stream):
    WHILE event_stream.hasNext():
        event = event_stream.next()
        processEvent(event)

        // Continuously drain top threats for the SOC dashboard
        WHILE priorityQueue.size > 0 AND priorityQueue.peek().risk_score >=
CRITICAL_THRESHOLD:
            topThreat = priorityQueue.extractMax()
            dispatchAlert(topThreat)
```

**6. Empirical Benchmark Harness (Hash Table Comparison):**

```
FUNCTION benchmark():
    load_factors = [0.25, 0.5, 0.7, 0.9, 0.99]
    distributions = ["uniform_random_ips", "skewed_zipf_ips"]  // simulate few malicious IPs
dominating

    FOR each dist IN distributions:
        dataset = generateSyntheticIPs(dist, N = 1_000_000)
        FOR each lf IN load_factors:
            FOR each strategy IN [ChainingHashTable, OpenAddressingHashTable]:
                table = new strategy(initial_capacity sized for lf)
                t0 = now()
                FOR ip IN dataset: table.insertOrUpdate(ip, dummyEvent)
                t_insert = now() - t0

                t0 = now()
                FOR ip IN sampleQueries(dataset): table.lookup(ip)
                t_lookup = now() - t0

                record_result(strategy, dist, lf, t_insert, t_lookup, collisions_count)

    plot_results()   // insertion/lookup time vs load factor, chaining vs open addressing,
uniform vs skewed
```

## Output Screen Shot:



```
============================================================
          REAL-TIME CYBER THREAT DETECTION ENGINE
============================================================
Total Events Processed : 6

----- DUPLICATE IP DETECTION REPORT -----
192.168.1.20    -> 2 occurrences
192.168.1.10    -> 2 occurrences

---- AVL TREE : EVENTS ORDERED BY RISK (Highest Risk First) ----
IP Address      | Risk | Event Type          | Timestamp
------------------------------------------------------------
172.16.0.8      | 100  | Malware Alert       | 10:00:20
192.168.1.10    | 95   | Brute Force Login   | 10:00:10
192.168.1.10    | 90   | Login Failed        | 10:00:01
192.168.1.20    | 85   | Port Scan           | 10:00:05
192.168.1.20    | 80   | Port Scan           | 10:00:05
10.0.0.5        | 20   | Normal Login        | 10:00:15

--- PRIORITY QUEUE : HIGH RISK EVENTS (Highest Risk First) ---
Rank | IP Address      | Risk | Event Type          | Timestamp
------------------------------------------------------------
1    | 172.16.0.8      | 100  | Malware Alert       | 10:00:20
2    | 192.168.1.10    | 95   | Brute Force Login   | 10:00:10
3    | 192.168.1.10    | 90   | Login Failed        | 10:00:01
...

----- HASH TABLE PERFORMANCE TEST (Insertion Time) -----
Number of Events | Insertion Time (Sec)
---------------------------------------
100              | ...
500              | ...
1000             | ...
2000             | ...
5000             | ...

---------------- COMPLEXITY SUMMARY ----------------
Hash Table (Search/Insert)       : O(1) average case
AVL Tree (Insert/Search)         : O(log n)
Priority Queue (Insert)          : O(log n)
Priority Queue (DeleteMax)       : O(log n)

Hash Table Size                  : 17 buckets
Hash Collisions                  : ...

------------------------------------------------------------

Program finished successfully.
```